

MIAE 212 · Rapid Review Sheet · Part 1

C++ Data Types, Sizes, Memory Limits & Numerical Traps

1. FUNDAMENTAL PRIMITIVE DATA TYPES

Type	Size (PC)	Size (Arduino)	Value Range	Precision / Notes
int	4 bytes	2 bytes	-2.14×10^9 to $+2.14 \times 10^9$	32-bit two's complement integer
float	4 bytes	4 bytes	1.2×10^{-38} to 3.4×10^{38}	~ 7 decimal digits (IEEE 754)
double	8 bytes	4 bytes	2.3×10^{-308} to 1.7×10^{308}	~ 15 – 17 digits (engineering standard)
char	1 byte	1 byte	−128 to +127 (unsigned: 0 – 255)	ASCII symbol code in single quotes ('a')
bool	1 byte	1 byte	true (1) or false (0)	In C++, 0 is false; any non-zero is true

- **Operator sizeof(T)** : Returns memory in bytes: `sizeof(int) = 4` , `sizeof(double) = 8` .

2. CRITICAL MEMORY & NUMERICAL TRAPS

- **Uninitialized Variables**: Reading `int x; int y = x + 1;` reads garbage memory bits! Always initialize: `int x = 0;` .
- **Integer Wrap-Around**: Adding 1 to max signed int wraps to negative: `2147483647 + 1 == -2147483648` .
- **The Integer Division Trap**: `int / int` truncates all fractional digits!
 - `1 / 3` evaluates to `0` !
 - `4 / 3 * 3.14 * r * r` evaluates to `1 * 3.14 * r * r` (loses 25% of true volume!).
 - **Fix**: Use floating literal: `4.0 / 3.0 * 3.14159 * r * r` .
- **Division by Zero**:
 - Integer: `1 / 0` triggers CPU crash / abort signal.
 - Floating: `1.0 / 0.0 == +inf` , `-1.0 / 0.0 == -inf` , `0.0 / 0.0 == nan` .
- **Round-Off & Cancellation**:
 - Adding tiny numbers to large ones loses precision: `1.0 + 1e-16 == 1.0` .
 - Machine epsilon: `float ≈ 10-7` , `double ≈ 2 × 10-16` .

3. TYPE MODIFIERS & MICROCONTROLLER ARCHITECTURE

- `const` : Read-only; cannot be mutated after declaration (`const double PI = 4*atan(1.0);`).
- `unsigned` : Discards negative values, doubling positive range (`unsigned int` : 0 to 4.29×10^9).
 - **Underflow Danger**: `unsigned int u = 0; u--;` produces `4294967295` .
- `short` : 2-byte integer (−32,768 to +32,767).
- `long` : 4 or 8 bytes.
- **Arduino Trap**: `sizeof(int) = 2` on 8-bit AVR microcontrollers! Overflows at 32,767.

4. FORMATTED CONSOLE I/O (<Iostream>)

```
#include <iostream>
using namespace std;
// Output in scientific notation with 10 decimal digits:
cout << scientific; cout.precision(10);
cout << "x = " << x << "\n";
// Revert back to fixed decimal format:
cout << fixed; cout.precision(4);
// Keyboard input:
cin >> val1 >> val2;
```